

Rethinking the Role of Network Stacks for Website Fingerprinting Defenses

A Case for Stack-Level Traffic Obfuscation

Paper Reference: ACM HotNets 2025 (Elisaveta Lavrentieva et al., University of Edinburgh)

Team Members: Aditya Shukla, Kartik Maheshwari

Advancing privacy through stack-level innovation.

What Website Fingerprinting does



Leaked Traffic Features

Bursts, bandwidth usage, packet sizes, flow sizes

The ISP observes traffic **Metadata** even when content is encrypted.

The Adversary Model

The Attacker: A passive network observer (e.g., ISP, National Firewall).

The Assumption: Closed-World Setting.

The attacker knows the user is visiting one of N (9 in this study) specific websites and has trained models on those patterns.

The Weapon: Deep Learning classifiers (k-FP, Var-CNN).

They analyze inter-arrival times (IAT) and packet sizes to identify traffic signatures.

Evaluating the effectiveness of defenses against this sophisticated **Adversary Model**.

Motivation

The Reality: Encryption (TLS 1.3, QUIC) protects *content*, but **Metadata** remains exposed.

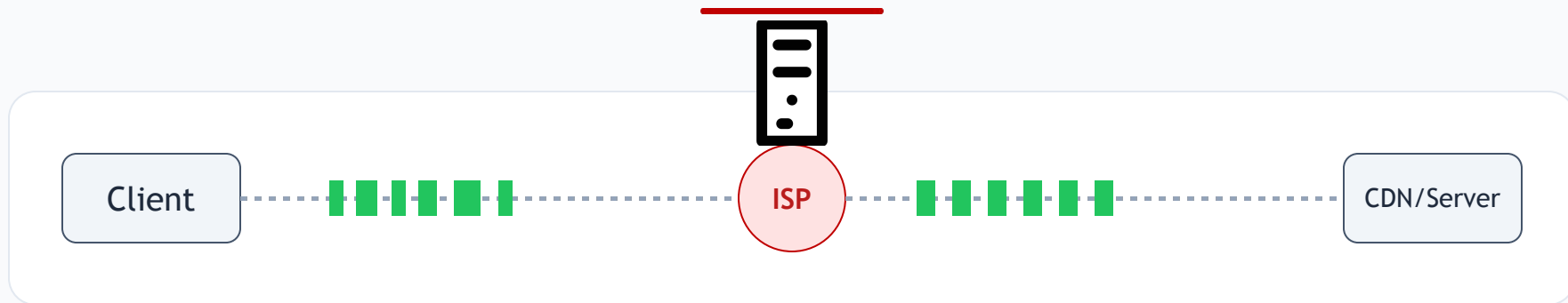
The Threat: Website Fingerprinting (WF) uses Machine Learning to analyze packet sequences (sizes, timing, direction).

The "Last Mile" of Privacy: With the rollout of ECH (Encrypted Client Hello), even the Server Name (SNI) is hidden.

Censors will now rely *entirely* on Traffic Analysis.

Protecting the users from WF seems crucial, but are the current defenses practical?

What the WF Defenses Want to Do



Padding

Adding "dummy" packets to hide the real volume of data.

Packet Size Mod

Changing packet boundaries to standardize size.

Timing Mod

Injecting delays to hide unique loading patterns.

THE GOAL: To make traffic patterns of different websites look identical to an observer at the ISP.

WF Defense Strategies

Regularization

Attempts to make packet sequences across different websites look exactly the same (e.g., fixed-size bursts at fixed intervals).

Example: BuFLO, RegulaTor

Obfuscation

Adds "noise" to features (size/timing) to confuse the Machine Learning classifier.

Example: WTF-PAD, FRONT

Target Abstractions

Tor-level

Defenses implemented in the Tor browser/relay.

TLS/QUIC-level

Defenses targeting direct HTTPS traffic.

Why App-Level Defenses are "Broken"

The Mismatch

Defenses are designed for **wire sequences** but implemented at the **application layer**.

Lack of Control

The application has no visibility into stack-level operations:

- **Congestion Control:** The stack may delay packets independently of application requirements.
- **Batching/Coalescing:** Small "obfuscated" messages are often combined into single large physical packets.

WF Defenses in Reality

Core Issue: Existing defenses operate strictly at the **application level**.

Inconsistent

The application lacks direct control over the network stack, leading to unpredictability.

- **No Guarantees:** The stack may reorder or delay packets, disrupting the intended defensive sequences.
- **Stack Autonomy:** Packet generation is managed by kernel-level logic, not app-level commands.

App-level obfuscation is *inconsistent*

Asynchronous Data Transmissions

Application transmissions do not align with wire sequences.

- **Send Buffering:** Packetization is based on PMTU, not application logic.
- **Ack-Clocking:** Transmissions are triggered by ACKs, not the app.

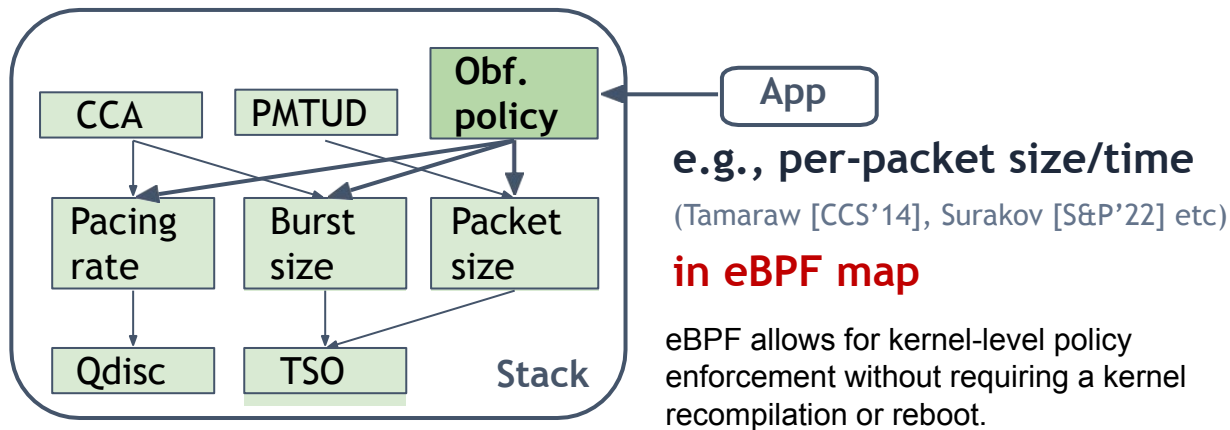
Stack-Level Optimizations

Kernel mechanisms disrupt timing and sequence obfuscation.

- **TSO:** Segmentation Offload creates micro bursts at line rate.
- **Scheduler:** Fair queuing and pacing delay packets independently.

Stob: The case for *stack-level* obfuscation support

- **New stack abstraction** for packet sequence obfuscation
- **Policy-driven:** App/admin creates and installs obfuscation policy
- **Integrated:** Cooperate with other decisions [1]



Stob Architecture: The Logic Flow

Step 1

App defines a
"Privacy Policy"
(e.g., a histogram of
desired packet sizes)

Step 2

Policy is pushed to an
eBPF Map
Kernel-level enforcement
without rebooting

Step 3

Transport Layer
queries the map
TCP/QUIC checks policy
before segmentation

Step 4

Dynamic TSO Sizing
Hardware segments
match policy, not just
MTU

Preliminary Experiment

Objective

Simulate a server-side kernel website fingerprinting defense to evaluate performance and efficacy.

Original



Split



Delayed



Preliminary Experiment

Investigate the Censorship Scenario

Evaluation of defensive capabilities against sophisticated website fingerprinting in restricted network environments.

Trace 1

C C

S S S

Trace 2

C C

S S S

C C

S S S

Trace 3

C C

S S S

C C

S S S

...

Performance Implications

Table 2: k-FP Random Forest accuracy rates. k-FP is distinguishing between 9 sites in a closed-world setting. N indicates the number of packets at the beginning of the trace on which the attack is applied.

N	Original	Split	Delayed	Combined
15	0.798 ± 0.017	0.825 ± 0.024	0.825 ± 0.030	0.795 ± 0.031
30	0.884 ± 0.007	0.860 ± 0.013	0.855 ± 0.030	0.850 ± 0.062
45	0.938 ± 0.016	0.897 ± 0.030	0.913 ± 0.021	0.904 ± 0.004
All	0.963 ± 0.002	0.980 ± 0.008	0.980 ± 0.014	0.992 ± 0.009

The "Hard" Problem: CCA Interplay



The Conflict

Privacy wants to delay packets to hide timing.

Congestion Control (CCA) wants to measure timing to calculate bandwidth.



The Risk

If Stob delays a packet, BBR or Cubic might think the network is congested and drop throughput to zero.



The Solution

Advocate for "**Privacy-Aware CCAs**".

Distinguish between "Intentional Delay" and "Network Congestion."

Summary and Research Agenda



Existing Defenses

Defenses are inconsistent or inefficient.

Stack-level support for traffic obfuscation is needed.



Deployment

How to incentivise CDN operators?

- Low performance overheads
- Differential privacy (NetShaper)



CCA Interplay

Avoid conflict with CCA transmit decisions.

Make CCA obfuscation aware.

Project: Implementing Stack-Level Defenses

An empirical reproduction of Stob's core thesis in a controlled network environment.



The Objective

To validate the "Application-Stack Gap" and demonstrate the real-world effectiveness of kernel-level traffic obfuscation against Website Fingerprinting (WF).



Our Approach

Build a complete, end-to-end WF evaluation pipeline to test a standard Linux network stack against a "Stob-defended" stack.



Technologies Used

Mininet: Network simulation environment.

eBPF: Kernel-level packet filtering and manipulation.

Python & Scikit-Learn: Data processing and machine learning analysis.

Experimental Setup: The Network Environment



Mininet Emulation

Simulates a high-fidelity Client-Server topology using virtual network namespaces to isolate traffic.

Key Configurations:

- 10Mbps bandwidth constraints
- Standard 1500 MTU enforcement
- Ensures natural packet boundaries
- Models realistic OS stack behavior



Traffic Gen & Capture

A dual-process pipeline designed to create and record diverse network datasets.

Components:

- **Mock Server:** 10 synthetic sites with unique fingerprints
- **Automated Crawler:** Systematic fetch routines
- **Capture:** Wire-level tcpdump on virtual interfaces
- **Fingerprints:** Varied file sizes and request counts

Defense Implementation: Stack-Level Regularization



Session-Level Volume Padding

Neutralizes the strongest WF feature: Total Volume.

- Server dynamically appends dummy data to every response.
- Pads all sessions to an identical target size (e.g., 30 MB).
- Eliminates volume-based site differentiation.



eBPF Kernel Timing Jitter

Destroys Inter-Arrival Time (IAT) fingerprints.

- Custom C program attached to kernel's TC egress hook.
- Injects 5ms base delay + 0-5ms random jitter per packet.
- Overrides OS scheduling to ensure wire-level obfuscation.

Regularizing traffic at the session and packet levels to bridge the Application-Stack Gap.

STOB Defense Design & Methodology

Defense (STOB)

Kernel-Level Traffic Obfuscation

Implemented via eBPF on the server's TC egress hook.

Volume Regularization (Padding):

- Dynamically appends dummy data to responses.
- Pads sessions to exactly 3 MB to neutralize byte count features.

Timing Obfuscation (Jitter):

- Overrides default OS schedulers via eBPF.
- Injects 5ms base delay + 0-5ms random jitter per packet.

Dataset Collection

Automated simulation of client/server fetches over Mininet, capturing raw traffic via tcpdump.

The Environment:

- Closed-world topology with 10 mock websites.
- Constrained 10Mbps/1500 MTU for realistic packet boundaries.

Clean Dataset (The Baseline):

- Undefended traces where stack efficiency leaks structural metadata.

The Attacker: ML Evaluation Pipeline



Feature Extraction

Parses PCAPs (via Scapy/tshark) to extract 7 vital features: incoming/outgoing bytes and packets, plus IAT mean, median, and range.



The Classifier

Random Forest algorithm (100 estimators) simulating the k-FP attack.

Model Training:

Training on clean and defended datasets.



Evaluation Modalities

- **Baseline:** Proves vulnerability of Linux stack.
- **Cross-Eval:** Tests if defense breaks fingerprints.
- **Adaptive:** Tests if smart attacker can learn new pattern.

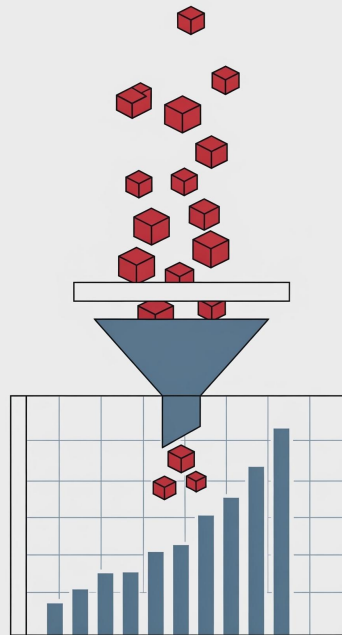
Feature Extraction & Traffic Analysis

Extracted Traffic Features

The evaluation pipeline extracts fine-grained features from both **Clean** and **STOB-defended** network traces:

- **Volume-Based:** Incoming and outgoing byte counts.
- **Packet-Based:** Total incoming/outgoing packet counts.
- **Inter-arrival Time (IAT):** Statistical analysis of packet timing including:
 - Mean
 - Range
 - Median

These features enable the Random Forest models to distinguish between different website fingerprints.



Evaluation Results & Interpretation

Model Performance Benchmarks

90%

CLEAN BASELINE

30%

STOB DEFENDED

10%

CROSS-EVAL

- **Clean vs. Clean:** High accuracy confirms traffic features are highly distinguishable.
- **STOB vs. STOB:** Accuracy drops significantly, approaching random chance.

Interpretation of Defense Strength

What strong defense looks like: Accuracy dropping toward random chance ($1/N$). The 10% cross-evaluation score suggests the model fails to find usable patterns in defended traces.

High accuracy risk: If accuracy remained high, it would indicate "Information Leakage" where the defense failed to mask unique fingerprints.

Conclusions & Future Directions

✓ Effectiveness of STOB

Significant Entropy: Accuracy dropped from 90% to 30%, effectively masking unique fingerprints.

Robust Defense: Cross-evaluation at 10% confirms that models trained on clean data fail to identify STOB-defended traffic.

Kernel Efficiency: eBPF implementation ensures minimal overhead while providing real-time obfuscation.

🚀 Future Experiments

- **Adaptive Jitter:** Testing dynamic timing variations based on real-time network congestion.
- **Scalability:** Evaluation across larger datasets (100+ websites) to measure classification decay.
- **Multi-Layer Defense:** Combining STOB with application-layer masking for "defense-in-depth."